

Walkthrough: choosing the right record

Use beside visual-review pages 1-2. Allow about 3 minutes for these two issues.

Open with: "We are reviewing whether the returned information means what we expect. The examples show possible failures; they do not mean we have found these failures in actual client records. We will ask the vendor to explain the results and choose how to address confirmed issues."

1. Hotel reservations: the wrong program

Say: "Think of a hotel checking an active reservation, then giving the key desk only the guest's name. The desk picks a newer canceled reservation. Our query can do something similar: the plan qualifies through one program, but another program record can supply its final details."

Point to the visual: The middle box: the handoff loses which reservation was checked. The last box shows why that matters.

Ask the team: Would we expect this plan to show the program that qualified it? Is there an intentional reason it would show a different one?

At the office: Filter PROGRAM_OUTSIDE_EXPECTED_SET. That locates blank or unexpected programs. A program inside the allowed list can still be the wrong enrollment; the vendor must compare the selection records.

2. Undated forms: the wrong plan can win

Say: "An office wants the newest form. One form is dated, and another has a blank date. The system puts the blank one first. A similar sorting rule can make an undated treatment plan beat a dated plan."

Point to the visual: The undated form moves to the top. There is no evidence that it is actually the newest.

Ask the team: What should happen when a plan has no begin date? What result would the receiving team expect?

At the office: Filter BLANK_RANKED_DATE. This finds missing dates on returned plans. It cannot show whether another eligible plan should have been selected.

Walkthrough: reading the results correctly

Use beside visual-review pages 3-4. Allow about 3 minutes, then agree what needs evidence.

3. Package labels: the value is there

Say: "A package has a postal code, but the warehouse software looks only for a field labeled ZIP. The value is present, but the system cannot find it. Our raw export names differ from the local demo combiner's expected names in the same way."

Point to the visual: The label and the software expectation. This is about how two systems communicate, not proof that the value itself is missing.

Ask the team: Who owns the agreed production file layout? Have we confirmed it with the actual receiving system?

At the office: LOCAL_DEMO_HEADER_DIFFERENCE is file-level INFO. It is expected for these raw exports. It does not prove a vendor contract violation or validate the full production mapping.

4. Restaurant orders: same item, different sides

Say: "Two tables order the same burger. One wants fries and one wants salad. The repeated burger is legitimate. We still need to know which side belongs to which table. A shared treatment goal can likewise have different objectives under different problems."

Point to the visual: The two separate paths. Repeating the goal is not automatically an error; losing its problem context is the concern.

Ask the team: Can our final combined plan preserve both branches? Who will confirm that using a real example?

At the office: Filter SHARED_GOAL_BRANCH, REPEATED_ENTITY_ID and PARENT_PATH_NOT_RETURNED. Compare the flagged IDs with the final combined plan. Do not remove rows just because the checker flags repetition.

Close with: "For each candidate, let's record what we expected, what was returned, and the vendor's explanation. We will distinguish a valid relationship, an export timing difference, and a confirmed defect before calling anything resolved."

At the office: run and read the checker

The kit runs locally on raw CSV exports. The README contains the full setup and limitations.

1. Prepare one export set. Save the six results with original headers as plans.csv, problems.csv, goals.csv, objectives.csv, interventions.csv and activities.csv. Use UTF-8 CSV. Keep all files from one run together; record query versions and export times.

2. Open PowerShell in the extracted kit folder. Use your office-approved PowerShell environment. No extra modules are needed. If local scripts are blocked, ask IT for the approved execution method.

```
.\Check-CareLogicResults.ps1 -InputFolder "C:\CareLogicReview\Exports"
```

A new report folder is created beside Exports.

3. Check completeness first. Open files.csv, then coverage.csv. CHECKED means that export check ran. NOT_RUN means prerequisites were missing. SOURCE_REQUIRED means final exports cannot answer that question.

4. Open findings.csv and filter by Check.

PROGRAM_OUTSIDE_EXPECTED_SET: inspect the returned program and ask which enrollment qualified the plan.

BLANK_RANKED_DATE: inspect the missing plan date and ask what other plans were eligible.

LOCAL_DEMO_HEADER_DIFFERENCE: confirm the actual production file agreement with its owner.

REPEATED_ENTITY_ID / SHARED_GOAL_BRANCH: trace each goal and its objectives under the correct problem.

PARENT_PATH_NOT_RETURNED / NO_PARENT_LINK: compare the full path across files and confirm whether a standalone relationship is valid.

ERROR entries: check incomplete files, missing IDs, repeated base rows or missing base matches before treating the finding as a source defect.

Locate a record: use File, SourceRecord and the client/document/entity IDs. SourceRecord counts data records after the header; it is not a physical line number when a field contains line breaks. A value of 0 means the finding applies to the file.

Keep the conclusion precise: ERROR identifies an export inconsistency or input problem. REVIEW needs context. INFO explains a format difference. No findings is not proof that the correct plan or program was selected. Those decisions still require source comparison.